

Library Catalogue REST API

1. Requirements

Functional Requirements

- **Books:** Create, Read, Update, and Delete (CRUD) book records.
- **Authors:** CRUD operations for authors.
- **Categories:** CRUD operations for book categories.
- **Search/Filtering:** Users must be able to filter books by author, category, or publication year.
- **Navigation:** API responses must provide links to related resources (HATEOAS).

Non-Functional Requirements

- **Security:** All modifying endpoints (POST, PUT, DELETE) require authentication via JWT (JSON Web Tokens). Read-only endpoints are public.
- **Performance:** GET requests must support caching to reduce server load.
- **Scalability:** The API must be stateless.
- **Usability:** Large datasets must be paginated. Errors must be verbose and use standard HTTP codes.
- **Format:** All data is exchanged in application/json

2. Data model

The system relies on three core entities:

1. Author

- **id (UUID):** Unique identifier
- **name (String):** Full name of the author
- **bio (String):** Short biography

2. Category

- **id (UUID):** Unique identifier
- **name (String):** Genre or category name

3. Book

- **id (UUID):** Unique identifier
- **title (String):** Title of the book
- **isbn (String):** International Standard Book Number
- **published_date (Date):** Date of publication
- **author_id (UUID):** Foreign key linking to Author
- **category_id (UUID):** Foreign key linking to Category

3. REST API Description

- **Base URL:** <https://api.library.com/v1>
- **Authentication:** Authorization: Bearer <JWT_TOKEN> header required for write operations.
- **Date Format:** ISO 8601 (YYYY-MM-DD).
- **Hypermedia (HATEOAS):** Responses include a `_links` object containing navigation URLs.

3.1. Pagination Strategy

Pagination uses `page` (0-indexed) and `size` (default 20) query parameters.

- **Request:** GET /books?page=1&size=10
- **Response:** Includes page metadata and HATEOAS links (first, prev, next, last).

3.2. Caching Strategy

- **GET Requests:** Include ETag and Cache-Control headers.
 - a) Cache-Control: public, max-age=3600 (Cache for 1 hour).
 - b) Clients verify freshness using If-None-Match: <etag>. If data hasn't changed, server returns 304 Not Modified.
- **POST/PUT/DELETE:** Do not cache. Successful write operations implicitly invalidate the cache for that resource.

3.3. Error Handling

Errors return a standard JSON structure.

```
JSON
{
  "error": {
    "status": 404,
    "code": "RESOURCE_NOT_FOUND",
    "message": "The book with ID 123 does not exist.",
    "details": "..."/>

```

4. API Endpoints & Operations

4.1. Books Collection

Resource: /books

4.1.1. List Books (GET)

- **Description:** Retrieve a paginated list of books.
- **Filters:** author_id, category_id, year.
- **Caching:** Yes (Cache-Control, ETag).

- **Response (200 OK):**

JSON

```
{
  "data": [
    {
      "id": "b1",
      "title": "Dune",
      "isbn": "978-0441172719",
      "_links": {
        "self": { "href": "/books/b1" },
        "author": { "href": "/authors/a1" }
      }
    }
  ],
  "page": {
    "size": 10,
    "totalElements": 50,
    "totalPages": 5,
    "number": 0
  },
  "_links": {
    "self": { "href": "/books?page=0&size=10" },
    "next": { "href": "/books?page=1&size=10" }
  }
}
```

4.1.2. Create Book (POST)

- **Description:** Create a new book.
- **Auth:** Required.
- **Body:** JSON object of the book.
- **Response (201 Created):** Includes Location header pointing to the new resource.

4.2.Single Book Resource

Resource: /books/{id}

4.2.1. Get Book (GET)

- **Description:** Retrieve details of a specific book.
- **Caching:** Yes.
- **Response (200 OK):**

JSON

```
{
  "id": "b1",
  "title": "Dune",
  "isbn": "978-0441172719",
  "published_date": "1965-08-01",
  "author_id": "a1",
  "category_id": "c1",
  "_links": {
    "self": { "href": "/books/b1" },
    "author": { "href": "/authors/a1" },
    "category": { "href": "/categories/c1" },
    "collection": { "href": "/books" }
  }
}
```

4.2.2. Update Book (PUT)

- **Description:** Full update of a book.
- **Auth:** Required.
- **Response (200 OK):** Returns updated resource.
- **Error (400 Bad Request):** Validation failure.

4.2.3. Delete Book (DELETE)

- **Description:** Remove a book.
- **Auth:** Required.

- **Response (204 No Content):** Empty body.

4.3. Authors Collection

Resource: /authors

- **GET /authors:** List authors (Paginated, Cached).
- **POST /authors:** Create author (Auth required).
- **GET /authors/{id}:** Get author details + links to their books.

4.4. Authentication Endpoint

Resource: /auth/login

- **POST /auth/login:** Exchange credentials for a JWT.
 - a) **Body:** { "username": "user", "password": "..." }
 - b) **Response (200 OK):** { "token": "eyJhGci...", "expires_in": 3600 }
 - c) **Error (401 Unauthorized):** Invalid credentials.

5. Status Codes

Code	Status	Meaning	Usage
200	OK	Success	Standard response for successful GET or PUT.
201	Created	Resource Created	Returned after a successful POST.
204	No Content	No Content	Returned after a successful DELETE.

Code	Status	Meaning	Usage
304	Not Modified	Cached	Returned when If-None-Match validates the ETag.
400	Bad Request	Client Error	Validation error (e.g., missing ISBN).
401	Unauthorized	Auth Error	Missing or invalid JWT token.
403	Forbidden	Permission Error	Valid token, but user lacks admin rights.
404	Not Found	Missing Resource	ID provided does not exist.
429	Too Many Requests	Rate Limit	User exceeded API rate limits.
500	Internal Server Error	Server Error	Unexpected failure on the server side.